

06 - MongoDB Sharding and Replication

CSCI 112 - Contemporary Databases

Maiko Francisco V Rugeria, MTM



Learning Objectives

- Understand the fundamentals of sharding and replication in MongoDB
- Get to know the components of a sharded MongoDB cluster and its overall architecture
- Understand the guidelines in choosing the best shard key for a collection

1

Scenario for the Day



Context Setting

- Jigu Telecom, The Coco Group's partner telecommunications company in South Korea, would like to migrate their website detection logs to MongoDB.
- Currently, an RDBMS type database holds their data on subscriber website access. It's beginning to create bottlenecks for their analytics related micro services, so they thought it was time to migrate to a NoSQL based database.

2

Replication



Replication: Review

- Can anyone tell the class what replication is for?
- Can anyone draw a diagram of a basic replication setup for databases?

3

Sharding

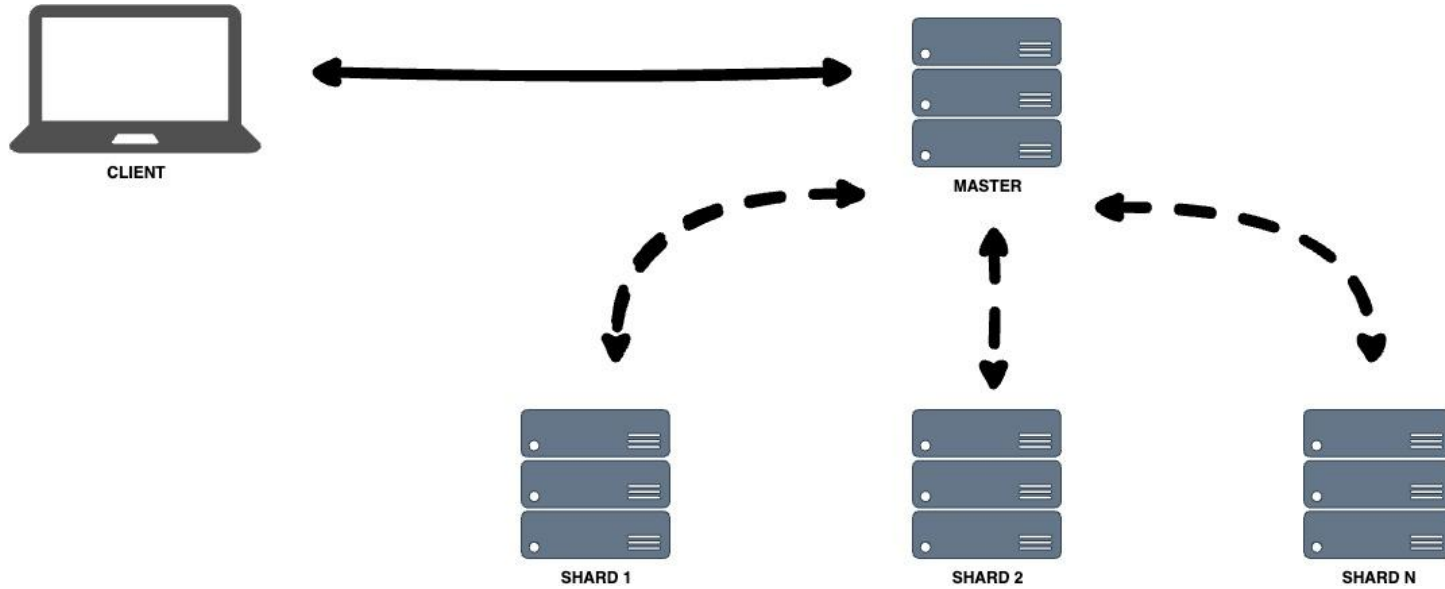


Sharding

- Sharding is a way to distribute documents in a collection over multiple servers (called a shard) to increase your resources.
- Sharding = scaling OUT (horizontal scaling)
- Replication compliments sharding by adding replicas to each shard for fault tolerance.

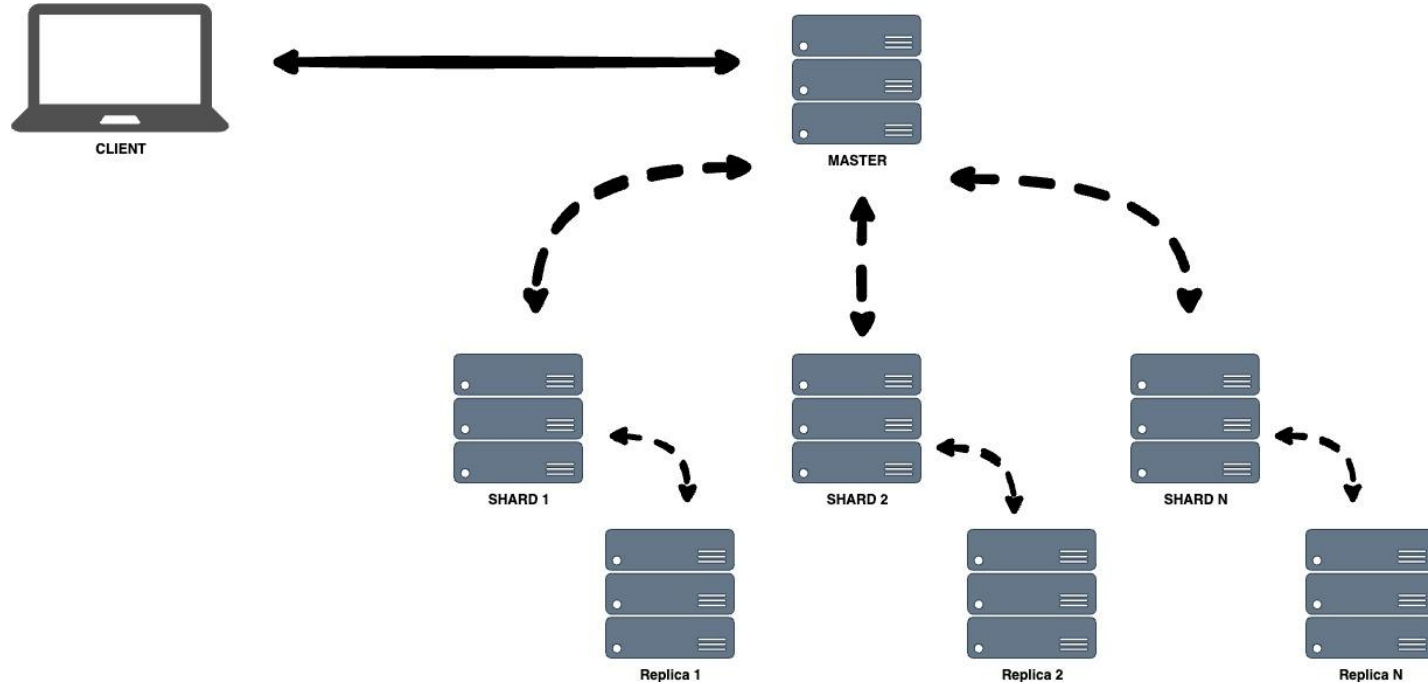


Sharding High Level Architecture





Sharding with Replication High Level Architecture





Why Do We Use Sharding?

- Primary reason is horizontal scaling – we can only scale vertically up to the max available server
- Geographical Segregation for:
 - ▷ Lower latency
 - ▷ Compliance (ex. GDPR)

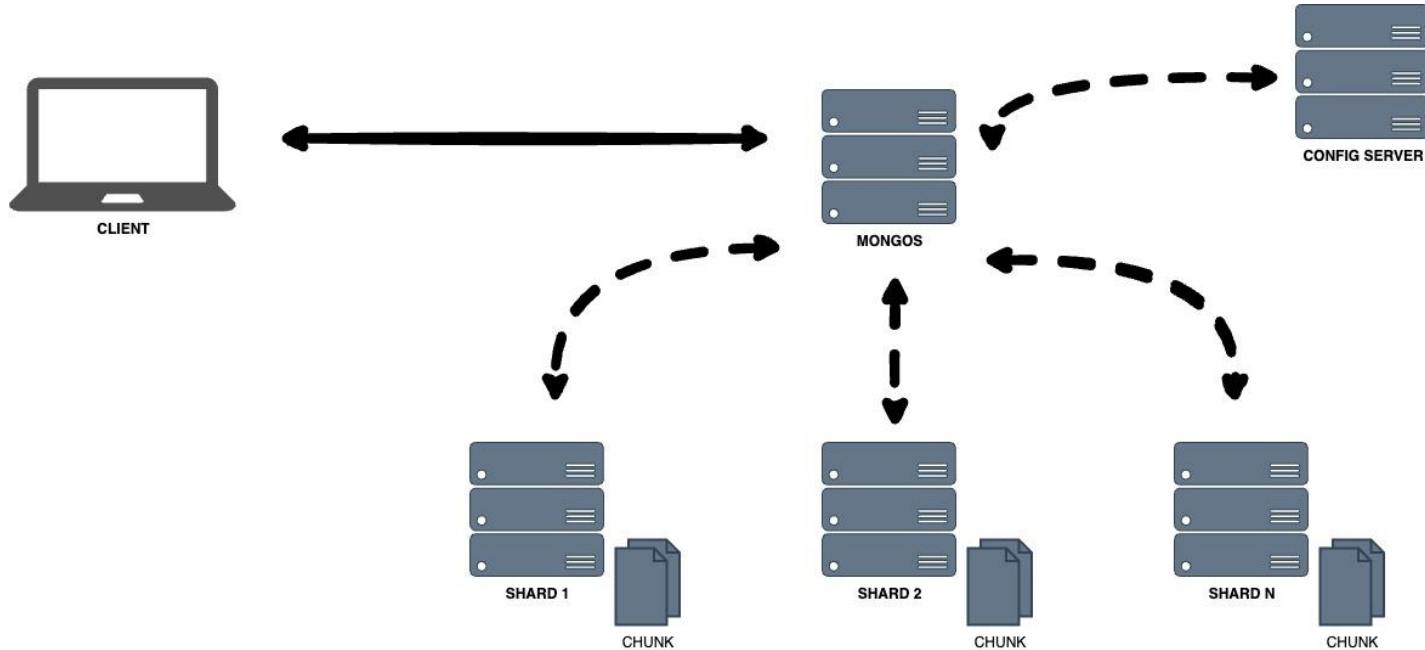


Sharding Components

- Chunk – a subset of the whole dataset
- Shard – the physical or virtual server that holds a chunk
- Config Server – holds the list of shards and chunk distribution
- Mongos – query router



Sharding Detailed Architecture



4

Sharded Collection Query Behavior



Querying Sharded Collections

- The same `find()` function is used to query a sharded collection. There are no changes in the way a query is executed from the client side.
- On the server side, the different shard components act together to perform the most optimized query path.

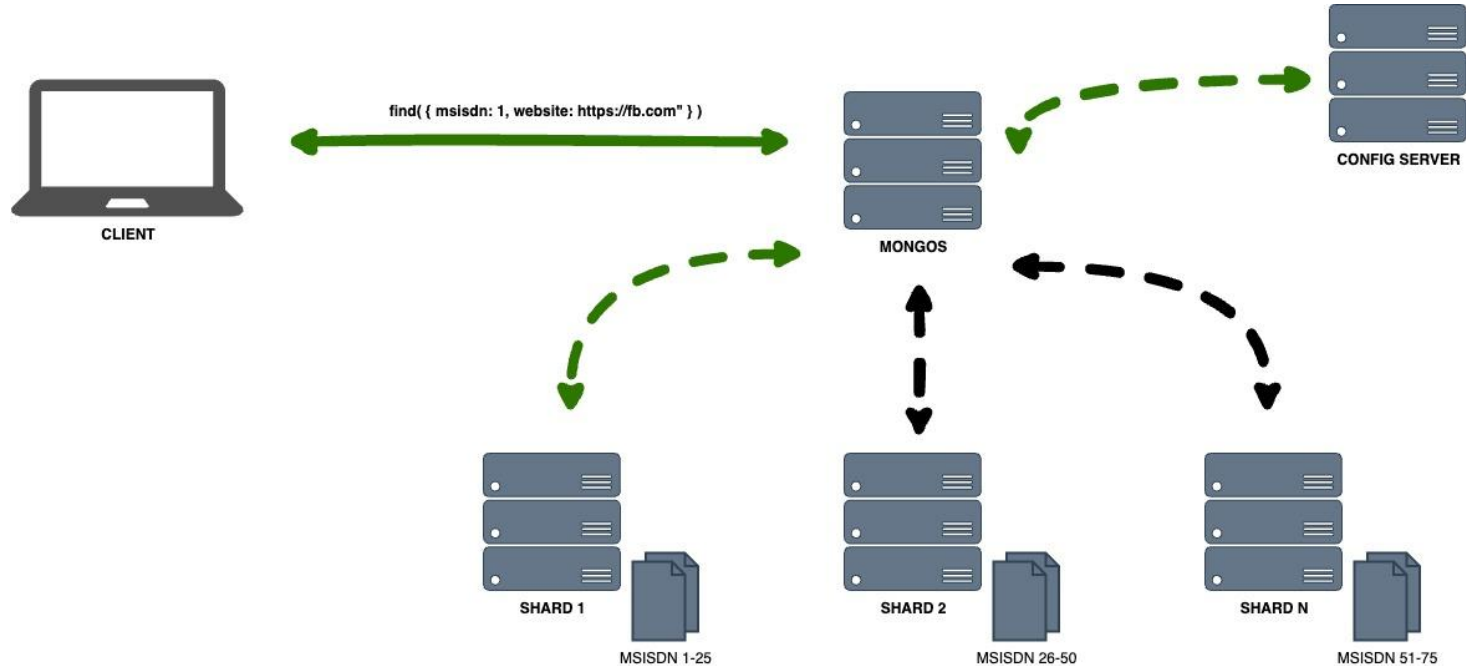


Jigu Telecom Website Detection

msisdn	website	timestamp	status
1	https://fb.com	2024-10-03 12:54:00	inactive
25	https://fb.com	2024-10-03 2:34:00	inactive
75	https://fb.com	2024-10-03 3:17:00	inactive
62	https://fb.com	2024-10-03 5:29:00	active

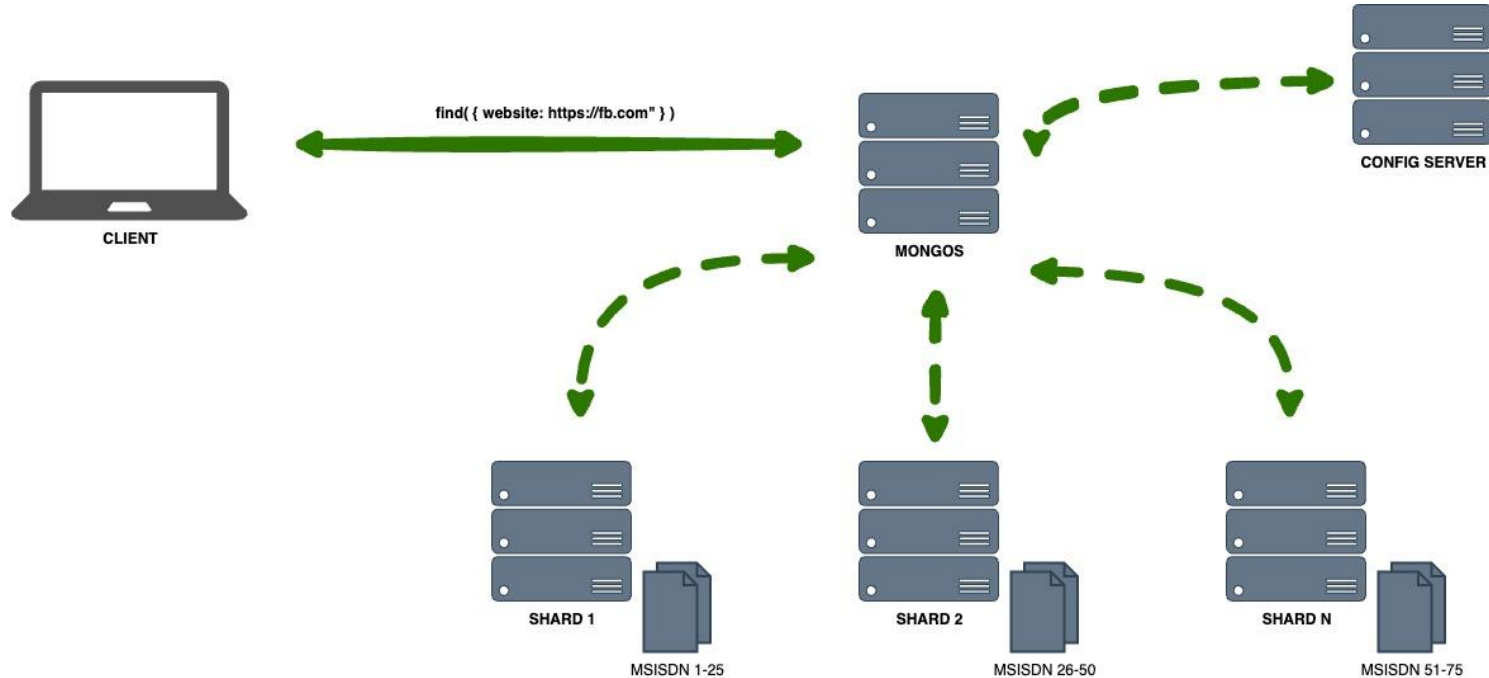


Good Query Behavior





Bad Query Behavior





Sharding Goal

- When sharding, the ultimate goal is to make your access patterns query the least possible number of shards.

5

Shard Keys

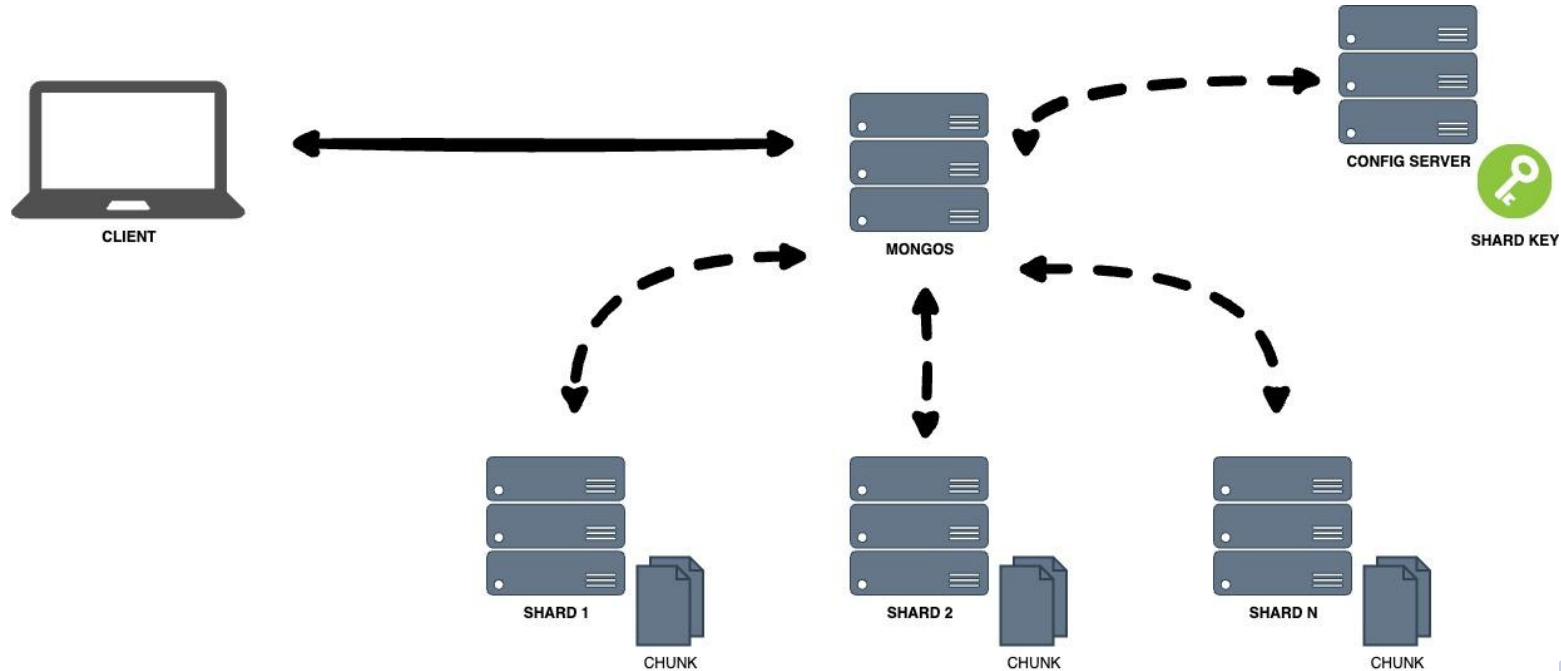


Shard Key

- A shard key decides where a particular document is stored.
- It can be one or multiple fields that come from the shared collection.
- Choosing the correct shard key can have a huge impact on the performance of your queries.
- Your access patterns will heavily influence which attribute should be used as shard key.



The Shard Key is in the Config Server





Choosing the Right Shard Key

- Must be high cardinality
- Used in majority of the access patterns (if not all)
- Even distribution of values
- Not continuously increasing in value
- Should be present in all documents
 - ▷ May not return documents that do not have the field
 - ▷ May perform a scatter gather
- For many-user systems:
 - ▷ Evaluate the data in the collection, does a record have an owner?
 - ▷ Choose the field that identifies the owner.



Many User System: Who is the Owner?

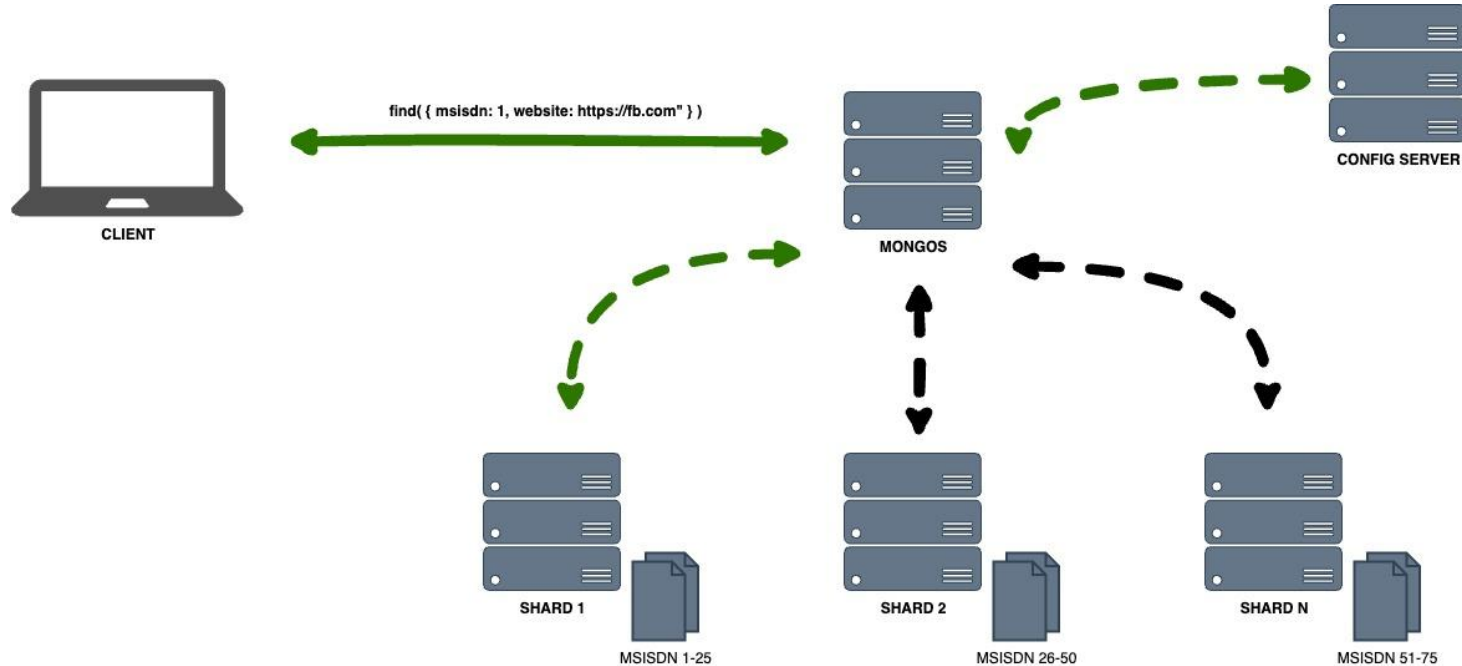
accountNo	transactionNo	amount
1	TX1	100
2	TX2	10902
3	TX3	76729
4	TX4	1110

6

Shard Key Distribution



Insert Operation Route in Sharded Clusters





How is Distribution of Shard Keys Chosen?

- MongoDB Chunk Management – by default, MongoDB chooses the distribution according to what it sees in the collection
- Manual Chunk Management – it is also possible for you to do the selection of chunk distribution, but it adds operational overhead.
- The general recommendation is to let MongoDB do the heavy lifting.



Shard Key Balancing

- Sometimes, due to unexpected volume of data and sometimes an incorrect selection of shard key, shard key distribution can become unbalanced.
- MongoDB automatically rebalances shard key distribution to optimize future queries.
- When a sharded collection rebalances frequently, you might have to choose a better shard key because rebalancing consumes compute and resources which could've been used for other more important workloads.



Sharding Distribution Types

- **Ranged** – Uses the range of values in the chosen shard key.
 - ▷ Best for: Queries using dates, numerical ranges, time series with the \$gte and \$lte operators
 - ▷ Not ideal for: Continuously increasing data –leads to hotspots
- **Zoned/Tagged** – Uses a location field as a shard key and distributes documents according to their location.
 - ▷ Best for: Geo-tagging, assigning tags to specific shards
 - ▷ Not ideal for: time series, dates, numerical ranges
- **Hashed** – Uses a hash value as a shard key for very high cardinality.
 - ▷ Best for: High write throughput, unbiased distribution
 - ▷ Not ideal for: Queries not involving the shard key



Ranged Sharding Distribution

- **Scenario:** You are setting up a sharded MongoDB cluster for your location detection logs. Your chosen shard key is timestamp simply because your access patterns would be based on timestamps.
- **Why this works:**
 - The queries will mostly be \$gte and \$lte on the timestamp key.
- **Risks:**
 - Hotspotting and continuous rebalancing
- **Mitigation:**
 - Use a compound key that can classify the timestamps like country or region



Zoned/Tagged Distribution

- **Scenario:** You are setting up a sharded MongoDB cluster for your subscribers collection. You are choosing between subscriber type and region to be your shard key.
- **Why this works:**
 - ▷ The queries will include either of the 2 attributes.
- **Risks:**
 - ▷ Queries must always have the shard keys, else scatter gather will be performed by MongoDB
- **Mitigation:**
 - ▷ Forecast the type of queries before choosing the shard key



Hashing

- A hash function is used to generate a fixed length string.
- This function is deterministic – the same input will produce the same output.

Value	Hash
1	c4ca4238a0b2237c825a1700f9c8fb6cb
2	098f6bcd4621d373cade4e832627b4f6
3	eccbc87e4b5ce2fe28308fd9f2a7b9b9



Hashed Distribution

- **Scenario:** You are setting up a sharded MongoDB cluster for your subscribers collection. But instead of zoning, your queries involve the `msisdn` more than any other attribute. This is why you chose the `msisdn` as the shard key.
- **Why this works:**
 - The hashed value will be evenly distributed across shards.
- **Risks:**
 - Queries must always have the shard keys, else scatter gather will be performed by MongoDB
 - Some sources say that hashing is bad when your documents exceed 10M (<https://youtu.be/NjvyFmuuRM4?si=k1hoHe0rhY2bTWbr> – Start at 20:00)
- **Mitigation:**
 - Use a compound shard key using the hashed value together with a zone or tag

7

Sharding Challenges



Sharding Challenges

- While sharding is a good practice to scale your platform horizontally, there are pitfalls:
 - ▷ Uneven distribution of data – commonly caused by shard key choice
 - ▷ Rebalancing of data – commonly caused by shard key choice
 - ▷ Additional latency – can't be avoided because of the distributed setup

End.

CSCI 112 - Contemporary Databases

Maiko Francisco V Rugeria, MTM